Assignment 6 - Visitor Pattern

Berdibek Meiirbek, SE-2528

1. Purpose and design

The program models a file system in memory. Three independent operations run on the same tree: total byte size, filename-and-size search, and statistics. No real files are read or written.

Element declares the common contract. **FileNode** stores a name and non-negative long size; **DirectoryNode** owns named children. The small abstract **Node** base shares name validation and parent bookkeeping, so ownership stays internal instead of exposing a public parent setter.

DirectoryNode.addChild() rejects duplicate sibling names, an existing parent, self-links and ancestor cycles before changing the tree. Names are case-sensitive single path segments. **getChildren()** returns an unmodifiable snapshot.

2. Double dispatch and traversal

First, **root.accept(visitor)** selects the actual element implementation. Inside that implementation, **this** has the concrete element type, which selects the matching overloaded visit signature. The actual visitor object then supplies that operation. Both concrete elements use:

```
public void accept(Visitor visitor) {
    Objects.requireNonNull(visitor, "visitor").visit(this);
}
```

Visitors own recursive traversal: their directory overload calls **child.accept(this)** once per child. Elements only dispatch, so they do not walk the children again. No instanceof, reflective checks or casts select behavior.

Start operations through **calculate(root)**, **search(root)** and **analyze(root)**. Each resets its state before dispatch, making visitor instances reusable between sequential runs. Direct accept/visit calls are traversal callbacks. Do not change the tree while traversing.

3. Demonstration and actual output

```
workspace/
  readme.txt (100 bytes)
  docs/
    report.txt (240), notes.txt (80)
    drafts/plan.txt (120)
  images/logo.png (500)
  empty/

Total size: 1040 bytes
Matches (.txt, >= 100 bytes):
  docs/drafts/plan.txt
  docs/report.txt
  readme.txt
Statistics: files=5, directories=5, maximum depth=3
```

4. UML class diagram

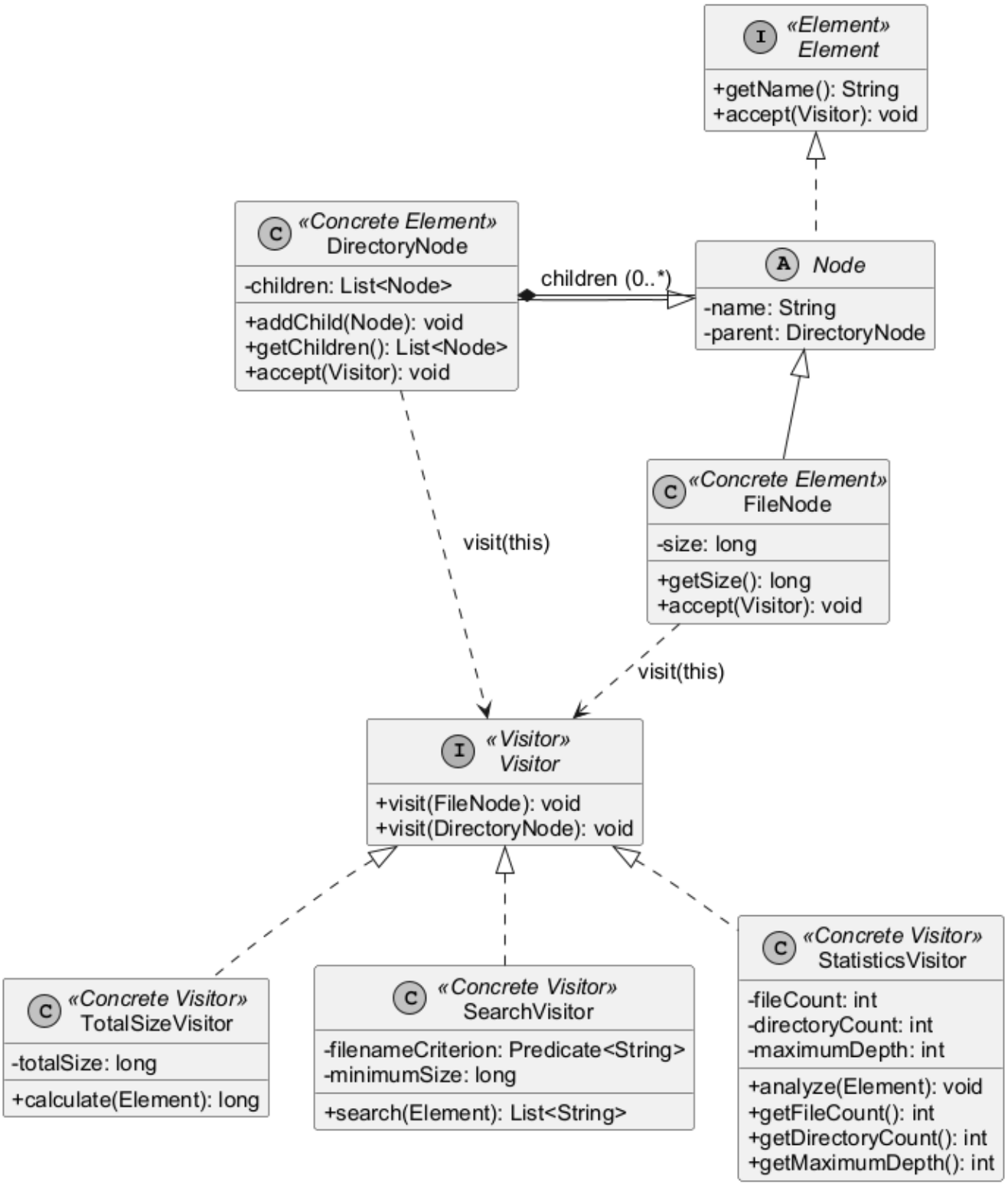


Figure 1. Rendered from **uml/visitor.puml** with PlantUML. Only relevant members are shown. Hollow triangles show inheritance/interface realization; the filled diamond shows directory ownership.

Paths: SearchVisitor accepts a configurable Predicate<String> and minimum byte size, combines both conditions with AND, and uses an inclusive size comparison. It keeps a directory prefix while descending and restores it when returning. Results omit the chosen directory root, use /, and are sorted lexicographically. A file root is reported by its own filename.

Depth: StatisticsVisitor begins at zero, increments depth before children and decrements afterward. Every element affects maximum depth, including empty directories. A directory root counts as one directory; a file root counts as one file at depth zero.

5. Five Clean Code practices

Meaningful naming: SearchVisitor.filenameCriterion and minimumSize directly describe the two search constraints.

Single Responsibility: Node/FileNode/DirectoryNode maintain the model and its integrity; TotalSizeVisitor, SearchVisitor and StatisticsVisitor each implement one separate operation.

Small methods: FileNode.accept() delegates in one statement, while getSize() and the statistics getters only return their requested value.

Encapsulation: DirectoryNode.getChildren() returns List.copyOf(children), and addChild() is the public mutation path that validates ownership, names and cycles.

No type-checking chains: FileNode.accept() and DirectoryNode.accept() call visit(this), letting overloaded visitor methods handle each type without if/instanceof branches.

6. Automated test results

Run from the Maven project root using **mvn test**. The JDK 17 run completed with **BUILD SUCCESS**: **24 tests, 0 failures, 0 errors, 0 skipped.** Test fixtures are separate from the demonstration data.

Test class (package filesystem)	Tests	Main verification
VisitorTest	12	T1-T4, T6-T7: overload dispatch, nested size, AND/inclusive filtering, sorted relative paths, empty/no-match cases, reuse.
TreeIntegrityTest	8	T8: invalid sizes/names, duplicates, multiple parents, cycles, snapshots, valid deep nesting.
StatisticsVisitorTest	4	T5-T7: file/directory counts, empty nodes, root-relative depth, repeated runs and later valid changes.

Additional checks cover zero-byte files, large long sizes, overflow rejection, a file as root, changed predicates, search result immutability and null inputs. Math.addExact prevents a silently overflowing total size.

7. Extension and trade-offs

Commit **fc6b24b** adds StatisticsVisitor, its tests and the client call after the initial size/search implementation. Element, Visitor, Node, FileNode, DirectoryNode and both accept methods are unchanged from commit **5fe0340**. A new operation only needs another implementation of the existing Visitor contract.

A new element type is more expensive: add its visit overload to Visitor and implement that overload in every visitor, alongside the new element's accept(). Visitor-owned recursion keeps the two elements small, but each operation repeats a short directory traversal loop. Recursive traversal also uses stack space proportional to tree depth; size/statistics are O(n), and search includes sorting its k matches: O(k log k).

8. Run and repository

```
mvn test
mvn -q compile exec:java
```

github.com/Mikosh001/Assignment_6_Visitor_Pattern

Reference: Refactoring.Guru - Visitor. Build reference: Apache Maven Surefire - JUnit Platform. OpenAI Codex assisted with implementation, documentation and verification.